



PROJET DE FIN D'ÉTUDES

Génération d'art algorithmique par modèles de langage : SVG et G-Code comme langages créatifs

ANDRIEU Ludovic
ANGLICHAU Jade
CURTIS Lucy
MAYOL Jean

Encadrant : **Guillaume Lagarde**

Mars 2026

1 Introduction

L'art génératif[10] est une pratique qui consiste à utiliser des algorithmes pour générer des œuvres en autonomie. De nos jours, elle est implémentée en majorité par des **modèles de génération d'image** qui opèrent directement sur les pixels. Une approche qui limite le contrôle sur la structure des œuvres générées.

L'art algorithmique[1], lui, est un type d'art génératif qui repose sur l'écriture de règles explicites, sous forme de code JavaScript ou Python, qui décrivent précisément la construction de l'œuvre. Ces règles peuvent être des tracés ou des structures géométriques par exemple. Cette méthode laisse beaucoup plus de contrôle sur les œuvres, cependant, l'œuvre obtenue est le résultat de l'exécution de code (par exemple un ensemble d'instructions) et non pas une image directement.



FIGURE 1 – Exemples d'œuvres art algorithmique

Notre projet consiste à explorer une approche alternative pour générer des œuvres qui combinerait code algorithmique et génération d'image, pour avoir à la fois le contrôle de la structure des œuvres et une génération directe d'image. Pour faire cela nous voulions voir s'il était possible d'utiliser un **modèle de langage** pour générer directement des œuvres d'art algorithmique qui seraient sous forme de fichiers vectoriels, tels que le Scalable Vector Graphics (SVG) ou le G-Code.

Le **SVG** est un format de fichier qui décrit les formes avec des formules mathématiques (lignes, courbes, polygones), contrairement aux images classiques (PNG, JPEG) qui sont composées de pixels. Son avantage étant que l'image peut être agrandie à l'infini sans perdre en qualité.

Le **G-code** est un langage de programmation pour les machines Computer Numerical Control ou CNC (imprimantes 3D, plotters). Il donne des instructions précises à la machine comme : où aller, à quelle vitesse et à quelle profondeur.

2 État de l’art

2.1 État de l’art au format SVG

Dans cet état de l’art pour le format SVG, nous nous sommes intéressés à deux aspects en particulier : le **traitement** de fichiers SVG et la **génération** de fichiers SVG, le premier étant moins répandu que le second.

2.1.1 Traitement des SVG

La grande majorité des modèles de génération d’image génèrent du contenu à partir d’une instruction (sous forme de texte) ou d’une image de référence. Certains modèles acceptent des fichiers au format SVG mais ils traitent généralement ces derniers comme des images rastérisées. Parmi les modèles qui « lisent » vraiment la structure vectorielle d’un fichier SVG, nous avons trouvé **Recraft AI**[14]. Afin d’évaluer ses performances, nous lui avons fourni en entrée deux images issues de notre jeu de données.

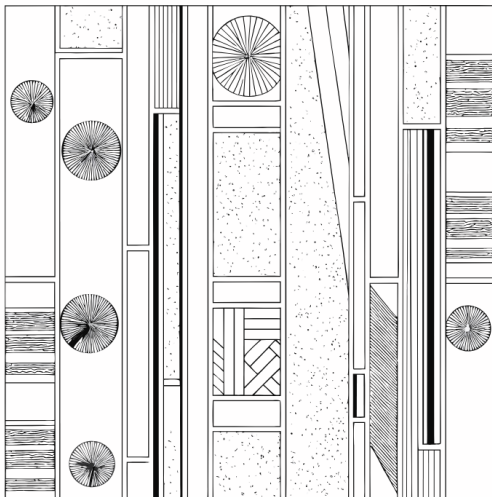


FIGURE 2 – Images donné à Recraft

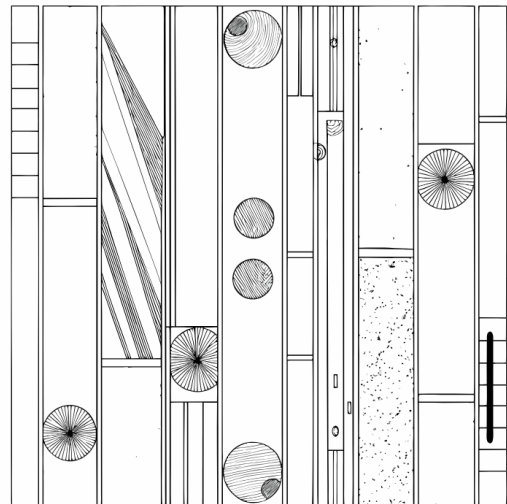


FIGURE 3 – Images généré par Recraft

Nous avons trouvé les résultats obtenus globalement satisfaisants. Ce modèle est le seul que nous ayons pu tester sur le traitement de fichiers SVG. En effet, de nombreux modèles sont payants ou restreignent l’accès à certaines fonctionnalités, ce qui a limité nos expérimentations. Nos tests de traitement de fichiers SVG ont donc uniquement pu être réalisés avec Recraft AI.

2.1.2 Génération des SVG

Les modèles capables de générer des fichiers SVG sont plus courants que ceux pouvant traiter ce type de fichier. Les deux principales architectures que nous avons identifiées sont le Image-to-SVG (image en entrée et fichier SVG en sortie) et le Text-to-SVG (instruction textuelle en entrée et fichier SVG en sortie).

Nous avons trouvé des modèles comme Freepik[6] et OmniSVG[5] qui exploitent ce format, ainsi que le modèle Punkericon[13], qui utilise gpt-image-1 d'OpenAI pour réaliser cette génération. De plus, le modèle Recraft AI, mentionné précédemment, permet également de générer des fichiers SVG.

2.1.3 Articles de recherche

Nous avons trouvé deux articles de recherche portant sur ce concept : DeepSVG[4] et RoboSVG[15].

DeepSVG traite le cas de transformation d'une entrée SVG vers une sortie SVG à l'aide d'un encodeur et d'un décodeur. Ce modèle se rapproche de l'approche de notre projet ; toutefois, il se concentre principalement sur des icônes et repose sur une représentation des fichiers SVG sous forme de commandes vectorielles.

Le papier RoboSVG compare différentes méthodes de génération de fichiers SVG. Inspiré de plusieurs modèles, dont DeepSVG, il aborde partiellement le cas de la transformation SVG vers SVG : à partir d'une image SVG incomplète fournie en entrée, le modèle est capable d'en proposer une complétion.

2.2 État de l'art au format G-Code

Nous avons adopté la même approche pour le format G-Code, en séparant le **traitement** et la **génération** de fichiers. Aucun modèle capable de traiter des fichiers au format G-Code n'a été trouvé. En revanche, nous avons trouvé plusieurs modèles capables de générer des fichiers dans ce format.

2.2.1 Génération à partir d'une image

Nous n'avons identifié qu'un seul modèle capable de générer des fichiers G-Code à partir d'une image : **G-Code Creator**[19] (proposé par yeschat.ai), qui utilise ChatGPT-4o pour générer des programmes en G-Code à partir d'images ou de fichiers PDF. Son objectif étant de convertir des structures géométriques complexes en instructions G-Code précises.

2.2.2 Génération à partir d'une instruction textuelle

Pour la génération de G-Code à partir d'une instruction textuelle, les seules options trouvées sont les modèles d'intelligence artificielle généralistes les plus utilisés aujourd'hui, comme ChatGPT, Claude ou Gemini. Ces modèles sont capables de générer des programmes en G-Code à partir d'une description textuelle (voir ci-dessous pour notre test avec ChatGPT).

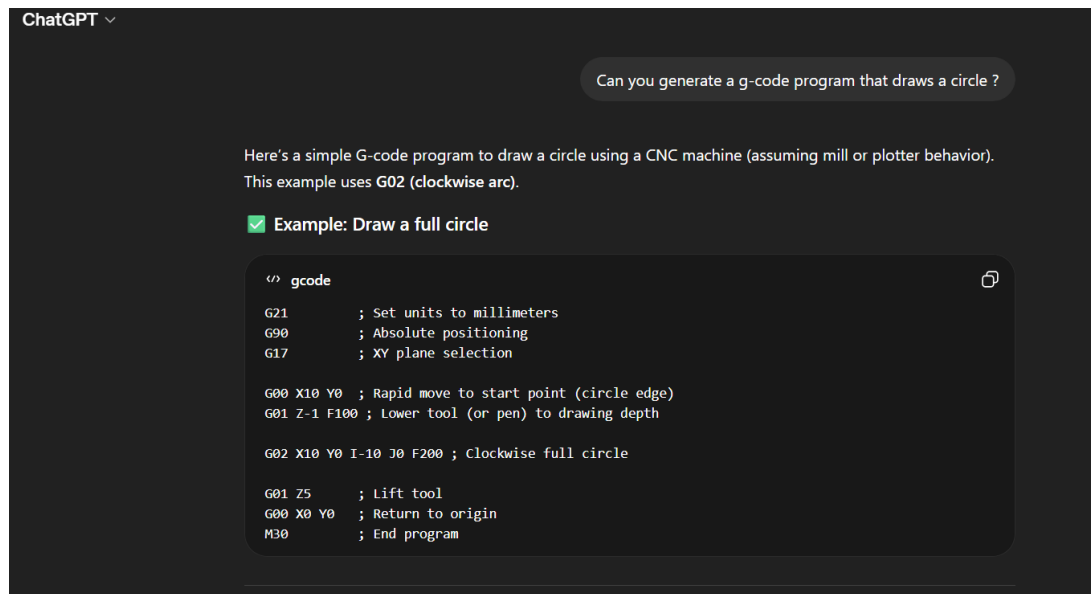


FIGURE 4 – Prompt G-Code et réponse générée par ChatGPT

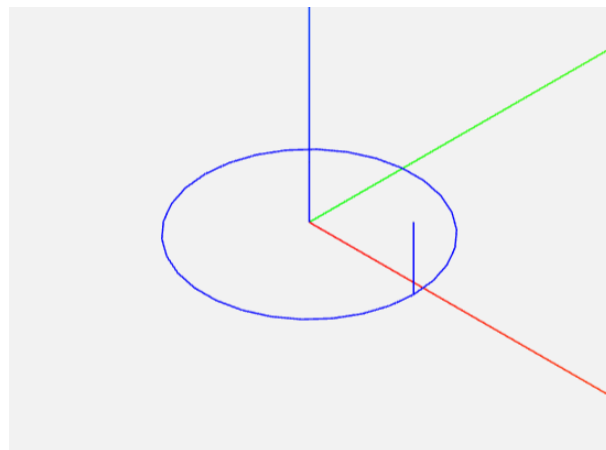


FIGURE 5 – Visualisation du G-Code généré par ChatGPT dans NCViewer

La figure obtenue correspondait bien à un cercle. Nous n'avons pas testé les autres modèles, en supposant que les résultats seraient similaires.

3 User stories

ID	Rôle
US-01	En tant qu’artiste , je veux générer des variations de mes œuvres déjà existantes, afin de visualiser les productions d’un modèle dans mon style d’art.
US-02	En tant qu’algorithme [17] (artiste numérique), je veux contrôler la complexité de l’œuvre générée afin d’adapter le résultat à mes besoins.
US-03	En tant que chercheur , je veux comparer les sorties du modèle entraîné sur du SVG vs du G-Code (convertis de l’SVG), afin d’analyser les différences entre chaque format.
US-04	En tant que développeur , je veux que le modèle génère des fichiers SVG syntaxiquement valides, afin qu’ils puissent être rendus sans correction d’erreurs.
US-05	En tant que traceur , je veux que la sortie respecte les contraintes physiques (lever et poser du stylo), afin que le tracé s’exécute proprement.
US-06	En tant qu’utilisateur , je veux une interface utilisateur en ligne de commande simple qui me permet d’entraîner un modèle.
US-07	En tant qu’utilisateur , je veux une interface utilisateur en ligne de commande qui me permet d’utiliser un modèle déjà entraîné pour générer des SVG et G-Codes.

4 Architecture

Le jeu de données sur lequel nous devons travailler est un ensemble de fichiers SVG qui ont été générés sur **Processing**[12] et dont le code a été fourni par notre client. Étant donné que l’on utilise des LLM, notre dataset est un ensemble de textes. Le format SVG étant un sous-ensemble du XML, notre dataset est bien au format texte, que nous avons normalisé.

Nous avons commencé le projet en explorant plusieurs possibilités de LLM qui pourraient répondre à nos besoins. Nous avons choisi d’utiliser une implémentation moderne en PyTorch de l’architecture GPT, disponible dans un projet open source : **NanoChat**[8].

4.1 Les datasets

Nous avons 3 datasets différents afin de pouvoir faire des tests par la suite. Ils sont constitués de 100, 500 et 1000 œuvres (fichiers SVG) générés grâce à l’outil **Processing**[12] à partir de codes Javascript fourni par notre client. Ce sont des images dans un style géométrique (voir exemples d’œuvres ci-dessous) que notre client souhaitait pour la génération.

Nos datasets étant des fichiers SVG, nous avons fait le choix d’avoir une option qui permet de les transformer en G-Code. Pour ce faire nous utilisons le plugin **vpype-code**[18] qui nous permet de transformer des SVG en G-Code via une ligne de commande. Nous avons désormais le choix d’utiliser un dataset de SVG ou de G-Code.

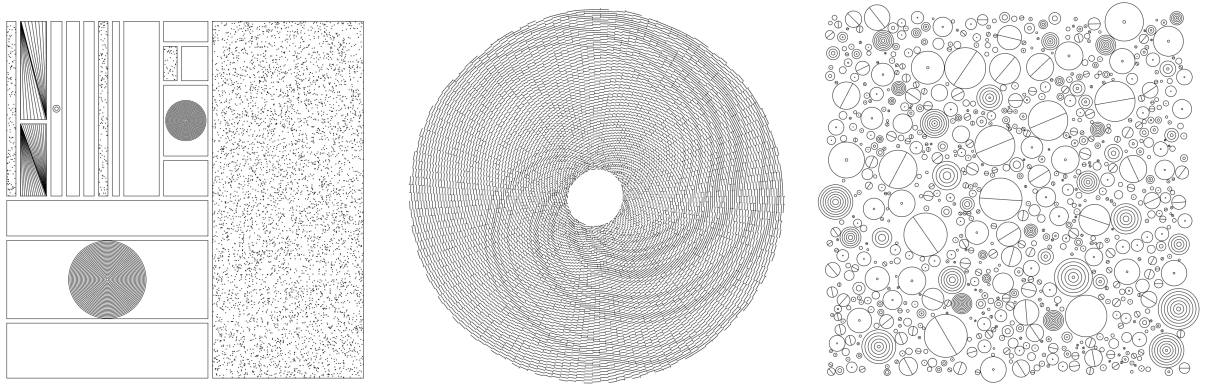


FIGURE 6 – Exemples d’œuvres du dataset

4.2 Structure du LLM

4.2.1 Pourquoi NanoChat ?

Notre projet ne nous imposait pas de technologies prédéfinies autres que d’utiliser un LLM. Nous avons donc commencé par envisager les différentes options pour arriver à notre fin : générer des SVG. Tout d’abord, nous nous sommes intéressés aux différents modèles open weights qui sont publiés afin de pouvoir les fine-tuner. Ces modèles ont un très grand nombre de paramètres qui très souvent dépasse le milliard, même pour les plus petits. Nous avons donc utilisé un **outil en ligne**[2] pour estimer les besoins en matériel requis pour entraîner ce genre de modèles. Avec cette méthode et en utilisant le plus petit modèle : Gemma3-270M, il aurait fallu 13h par époque au mieux pour l’entraîner sur le jeu de données de 100 SVG au CREMI, ce qui n’était pas envisageable.

Nous nous sommes donc tournés vers des modèles beaucoup plus petits que l’on pourrait entraîner plus facilement. Notre client nous a orientés vers un LLM pouvant être utile à notre projet : [NanoGPT](#). En lisant la documentation, nous nous sommes aperçus que le répertoire datait et n’était plus maintenu à jour. La documentation mentionnait **NanoChat**[8], bien plus récente. En regardant NanoChat, nous avons pu constater quelques avantages à l’utiliser pour notre projet. Après un test sur un dataset de SVG autre que celui du projet, nous avons pu voir les performances et la simplicité d’utilisation du modèle, ce qui nous a confirmé ce choix.

4.2.2 Structure de NanoChat

L’architecture de LLM que nous utilisons est celle de NanoChat, qui est inspirée de l’architecture GPT. NanoChat apporte toutefois des modifications afin de à la fois la moderniser, mais aussi de permettre de la miniaturiser et de permettre un entraînement plus efficace. Elle est donc basée sur des Transformers avec une self-attention causale et seulement le décodeur, et utilise un encodage rotatif (RoPE). Chaque bloc fonctionne de la manière suivante :

$$x \leftarrow x + \text{attention}(\text{RMSNorm}(x))$$

$$x \leftarrow x + \text{MLP}(\text{RMSNorm}(x))$$

Pour le MLP, c’est très basique, il s’agit d’un simple Linéaire(expansion $\times 4$) $\rightarrow$ ReLU $\rightarrow$ Linéaire(reviens à la dimension de base). Le mécanisme d’attention est moins trivial. Voici les formules (que nous expliquerons par la suite) :

$$q, k, v \leftarrow \text{LinearProjection}(x)$$

$$v \leftarrow v + \text{ValueEmbed} \times 2 \times \text{sigmoïde}(\text{LinearGate}(x))$$

$$q, k \leftarrow \text{RMSNorm}(\text{RoPE}(q, k))$$

$$y \leftarrow \text{LinearProjection}(\text{FlashAttention}(q, k, v, \text{causal}=\text{True}))$$

Tout d’abord, nous calculons les queries, keys et values avec une projection linéaire, puis un gating est utilisé pour ré-ajouter de façon conditionnelle la *ValueEmbed* (embedding initial des tokens). Ensuite, l’embedding positionnel est ajouté sur les keys et queries avec RoPE, ce qui permet de ne pas avoir de

paramètres à apprendre et de ne pas utiliser de dimensions dans x pour la position. RoPE est utilisé dans les grands LLM, notamment dans LLaMA par exemple. Vient ensuite le mécanisme d'attention utilisé : *Flash Attention*. Il s'agit d'une implémentation qui permet de grandement réduire l'utilisation en mémoire car, au lieu d'utiliser les matrices d'attention complètes en une fois, elle les découpe par blocs, ce qui permet de garder une utilisation en mémoire non plus quadratique mais linéaire. Cette technique permet aussi de réduire les FLOPs en nous laissant le choix d'utiliser sur certains layers un contexte réduit pour les tokens. Dans NanoChat, on peut choisir entre Long et Short, qui correspondent respectivement à 100% et 50% de contexte, et impose que le dernier layer soit toujours en Long contexte. Le contexte Short permet donc de réduire par 4 les FLOPs de l'attention pour les layers qui l'utilisent. Nous avons fait le choix d'utiliser la configuration suivante : SSSL (pour plus de 4 layers, on boucle sur cette séquence).

4.3 Interface en ligne de commande (CLI)

4.3.1 CLI d'entraînement

La demande de notre client sur le produit final était d'avoir une interface en ligne de commande (CLI) qui permette de lancer une pipeline d'entraînement d'un transformer sur un dataset donné. Le tout avec possibilité de configurer les paramètres principaux (taille du modèle, type de tokenizer...). Lors de l'exécution de ce CLI, on regarde avant tout si le chemin du dataset est valide ou non. Puis on regarde le tokenizer demandé et on le charge si possible. Si le nom est invalide, on demande à l'utilisateur si il souhaite qu'on en entraîne un nouveau à partir du dataset mis en entrée. Ensuite on tente de définir le modèle à l'aide d'une variable globale déjà définie, où l'on stocke les paramètres nécessaires de chaque modèle pour l'entraînement. On l'entraîne et on affiche les métriques. Le modèle est automatiquement sauvegardé dans un dossier. Un modèle contient plusieurs checkpoints avec ses métriques différentes et chaque checkpoint correspond au nombre d'époques sur lequel le modèle s'est entraîné. C'est avec cet outil que nous avons entraîné chaque modèle.

4.3.2 CLI de génération

La demande devait aussi pouvoir permettre de générer des SVG avec le transformer entraîné. Pour cela, on demande à l'utilisateur des paramètres essentielles comme le chemin du dataset, le nom du fichier à sauvegarder et le nom du modèle utilisé avec sa version. L'utilisateur a aussi le choix de mettre ou pas des limites sur la génération comme le temps, le nombre maximum de tokens à atteindre, le top_k et si il y a un début de SVG. En cas de début de SVG, il va générer la suite du fichier SVG mis en entrée. Pour cela on charge le modèle souhaité, et on génère dans un fichier le SVG en question sous forme de texte (dû à une mauvaise nomenclature de la génération). Ce n'est qu'à la fin de l'exécution du CLI que l'utilisateur peut consulter le SVG. Ces fichiers sont regroupés dans un dossier. Pour visualiser le SVG, on copie le texte et on le lit grâce à un viewer de SVG en ligne comme `svgviewer` [16].

4.4 Les métriques

4.4.1 Weights & Biases

Pour afficher les métriques, nous utilisons un logiciel extérieur : Weights & Biases (wandb) [3]. Ce logiciel permet d'afficher les métriques avec les courbes que l'on souhaite, peut faire un récapitulatif de chaque valeur sous forme de tableau. Cela est indexé dans des runs du projet sur lesquels nous pouvons avoir le contrôle (noms, ID, métriques, commentaires éventuels). Pour cela, nous avons fait quelques fonctions sur le projet : une pour afficher les métriques d'une run, une pour faire une nouvelle run avec de nouvelles métriques, et une pour mettre à jour une run existante en ajoutant des nouvelles métriques. Ces fonctions sont utilisées pour que, lors de l'entraînement du modèle, si le modèle n'existe pas, on crée une nouvelle run sur wandb et on affiche les métriques de cette run. Et si le modèle existe déjà, on reprend les métriques du modèle et on en crée de nouvelles pour la suite d'entraînement d'un modèle.

4.4.2 Liste des métriques

Voici la liste des métriques utilisées pour ce projet :

- **CE** : la cross-entropie. Mesure de l'incertitude du modèle à partir des probabilités des tokens prédits pour les tokens dans le SVG qui doit être généré. Une valeur plus petite signifie que le modèle prédit mieux.

- **CE2** : utilise la cross-entropie mais sur les tokens les plus probables que le modèle prédit au lieu d'utiliser le texte original des SVG à prédire. Il s'agit d'une métrique non conventionnelle mais que l'on a trouvé intéressant à mesurer pour notre compréhension des LLM.
- **ENTROPY** : mesure l'entropie sur tous les tokens prédits, pas seulement sur les tokens ciblés. Plus elle est proche de CE et moins le modèle a de chance de faire des erreurs aléatoirement.
- **BPC** : une mesure basée sur la CE qui estime le nombre de bits utilisés pour encoder un caractère.
- **PPL** : la perplexité du modèle basée sur la CE. Estimation du facteur de branchement du choix du token suivant. Plus la valeur est proche de 1.0, moins le modèle hésite et choisit le bon prochain token.
- **PPL2** : la perplexité du modèle basée sur la CE2. Reflète le facteur de branchement réel du modèle, et non sa capacité à choisir le bon token à prédire.
- **LOGITS SD** : mesure l'écart-type dans les valeurs des logits prédits par le modèle. Sert à mieux comprendre les LLM.
- **TOP-1** : vérifie si le bon token correspond au token prédit comme le plus probable par le modèle.
- **TOP-5** : vérifie si le bon token se trouve parmi les 5 tokens prédits comme les plus probables par le modèle.
- **Epoch duration** : durée totale de chaque époque.

5 Réalisation

5.1 Génération et chargement du dataset

Au démarrage, le dataset parcourt un répertoire et charge tous les fichiers .svg trouvés, dans l'ordre alphabétique. Chaque fichier est lu puis passé dans `clean_svg()`, qui utilise le parser XML (de lxml) pour re-sérialiser le contenu de manière normalisée (les commentaires et les espaces superflus sont supprimés) et la structure XML est standardisée. Ça garantit que deux fichiers SVG sémantiquement identiques mais formatés différemment produiront exactement la même séquence de tokens. (NOTE : jsp si intéressant à dire). Si le paramètre `use_gcode=True` est activé, chaque SVG est converti en G-code via l'outil en ligne de commande `vpype`. Les coordonnées sont normalisées dans l'espace machine (9.375×9.375 unités).

5.1.1 Tokenisation

Chaque texte (SVG ou G-code) est d'abord encadré par deux tokens spéciaux :

```
<|output_start|> ... <|output_end|><|output_end|>
```

Le double token de fin est intentionnel : il apprend au modèle que lorsqu'il génère un premier token de fin, le suivant doit également être un token de fin (stabilise la terminaison de la génération). Le texte ainsi préparé est ensuite tokenisé par un tokenizer de sous-mots de type BPE (Byte-Pair Encoding), chargé depuis un fichier `.json` au format HuggingFace. Contrairement à une tokenisation caractère par caractère, le BPE regroupe les séquences fréquentes en tokens uniques. Par exemple, des fragments récurrents dans les SVG comme `fill="`, `<line` ou dans le G-code comme `G00 X` seront encodés comme un seul token plutôt qu'une suite de caractères individuels. Cela réduit beaucoup la longueur des séquences.

5.1.2 Système de découpage en chunks

Un problème que nous avons rencontré était que les fichiers au format SVG sont très grands, et dans notre cas, les séquences tokenisées étaient plus longues que la fenêtre de contexte, `context_size`, du modèle. Nous avons donc la fonction `chunk_tokens()` qui découpe les séquences en "chunks" de taille `context_size + 1` avec un pas de glissement de 50% (`context_size // 2`).

Par exemple, pour `context_size = 8` et une séquence de 20 tokens :

	Tokens	Positions
Séquence	[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19]	
Chunk 0	[0 1 2 3 4 5 6 7 8]	0 → 8
Chunk 1	[4 5 6 7 8 9 10 11 12]	4 → 12
Chunk 2	[8 9 10 11 12 13 14 15 16]	8 → 16
Chunk 3	[12 13 14 15 16 17 18 19 -1]	12 → 20 (paddé)

Le chevauchement de 50% garantit que chaque transition entre tokens est couverte par au moins un chunk complet, y compris les transitions qui tomberaient exactement sur une frontière sans chevauchement.

Le dernier chunk d'un fichier étant souvent plus court que `context_size + 1`, il est complété avec la valeur `IGNORE_INDEX = -1` jusqu'à atteindre la taille attendue. Lors du calcul de la loss, ces positions sont explicitement ignorées grâce au paramètre `ignore_index=-1` de `CrossEntropyLoss`. Le modèle n'est donc jamais pénalisé pour les positions de padding

5.2 Difficultés rencontrées

La principale difficulté rencontrée au cours de ce projet a été le temps. Certains entraînements pouvant durer jusqu'à trois jours, le délai d'un mois et demi a été trop court pour obtenir des résultats particulièrement significatifs.

D'autres difficultés ont également été rencontrées, notamment liées à l'entraînement sur GPU, qui constitue une contrainte à la fois matérielle et temporelle. Nous n'avions soit pas de GPU, soit de GPU de performances moyennes, rendant les entraînements relativement lents. Pour comparer, le GPU RTX 4000 (disponible dans une salle du CREMI) était environ 1,5 à 2 fois plus rapide que des GPU RTX 3060 ou RTX 4060. Pour pallier ce problème, nous avons fait certains entraînements sur nos machines personnelles, ainsi qu'au CREMI, dans la salle 104 équipée de GPU RTX 4000.

Nous avons également rencontré des difficultés lors des entraînements à distance via SSH, en raison de déconnexions inattendues.

Enfin, compte tenu des contraintes de temps, nous n'avons pas pu entraîner un modèle suffisamment performant pour obtenir des résultats optimaux. Nous avons donc privilégié l'entraînement de plusieurs modèles afin d'analyser leurs performances respectives et d'identifier les paramètres les plus adaptés.

5.3 Tests

5.3.1 Tests unitaires

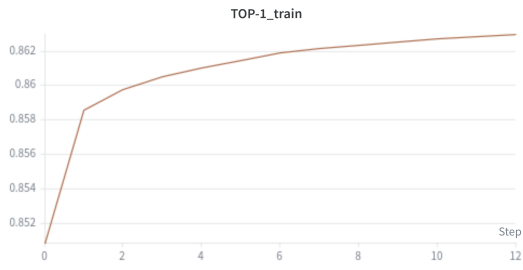
Pour les tests, nous les avons regroupé dans des jupyter notebooks : chaque fichier .py a son notebook de test correspondant et toutes les fonctions nécessaires pour le projet sont testées. Par exemple le fichier `historique.py` sert à mettre à jour les informations des métriques et de potentiels commentaires dans des dictionnaires, utilisés pour l'affichage sur wandb. Les tests réalisés permettent de vérifier que toutes les fonctions utilisées pour cela sont correctes et font bien ce qu'on recherche à faire. Il existe aussi un notebook de test *firstTestWithNanochat*, qui n'utilise pas de fonctions développées pour le projet. En revanche ce test utilisait des fonctionnalités de nanochat pour voir si cela fonctionnait pour les performances et la pertinence du modèle par rapport à l'intégrité de notre projet. Cela nous a aussi permis de comprendre comment aborder correctement le projet, c'est-à-dire de bien réaliser la tokenisation, le chargement de dataset et la gestion des métriques avant de faire l'entraînement des modèles et la génération de SVG ou de G-Code. Les tests ont permis aussi de voir les performances des fonctionnalités de nos modèles, très important du fait que pour un entraînement aussi lourds, tout gain de temps est bénéfique. Les tests *experimentWithTraining* et *experimentWithGenerate* ont été importants, ils nous donnaient l'information de base du temps d'entraînement du modèle, des paramètres pour le modèle, de la vitesse de génération mais aussi du fonctionnement de la génération (voir si le vocabulaire renvoyer était bien sur le bon format).

De même pour la partie dataset et LLM avec entraînement et génération, tout les tests ont pu être réalisés dans les notebooks associés.

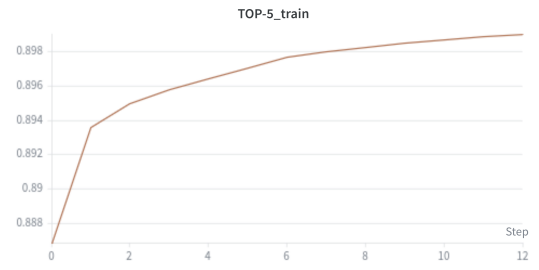
6 Résultats

6.1 Pertinence et efficacité

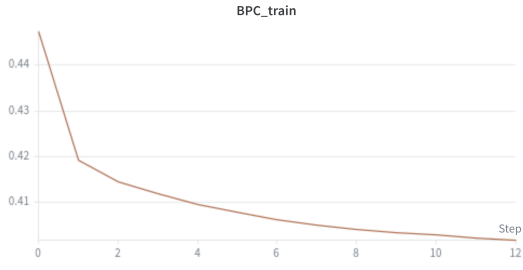
Afin d'analyser la pertinence et l'efficacité de nos modèles, nous utilisons les métriques présentées précédemment (section 4.4.2). Les figures suivantes illustrent l'entraînement d'un modèle SVG de 11,8 millions de paramètres sur le dataset de 1000 fichiers SVG, sur 12 époques. L'ensemble de ces graphes représente donc les performances et la pertinence ou non de notre modèle.



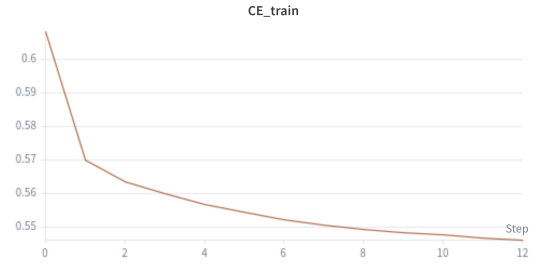
(a) top 1



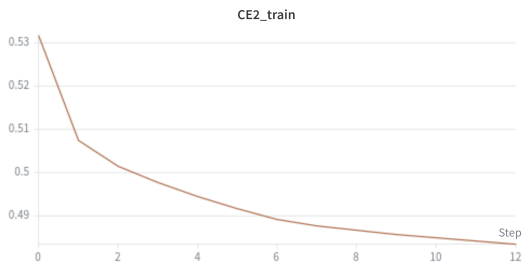
(b) top 5



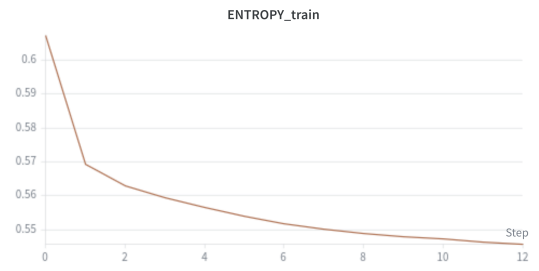
(c) bpc



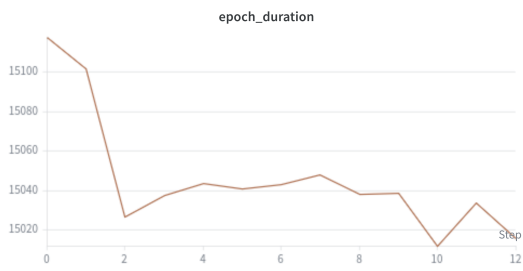
(d) ce



(e) ce2



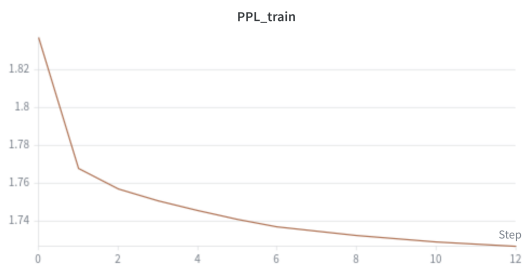
(f) entropy



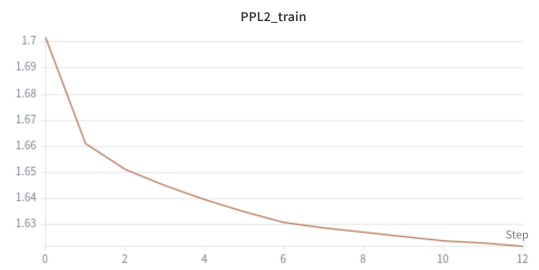
(g) epoch duration



(h) logits sd



(i) ppl



(j) ppl2

FIGURE 7 – Métriques d'entraînement d'un model SVG de 11M.8 de paramètres sur le dataset de 1000 SVG, avec 14 époques au total

model & params	depth	dims	context size	heads	vocab size	Mflops/token*	Gflops**
1.6M	6	128	4096	1	585	58	237
1.6M_ext	6	128	16384	1	585	209	3'423
5.5M	6	256	4096	1	585	130	532
5.5M_ext	6	256	16384	1	585	432	7'077
6.0M	7	256	4096	2	360	147	602
11.8M	6	384	4096	1	585	216	885
20.5M	6	512	4096	1	585	317	1'297

TABLE 1 – presets utilisés pour entraîner les modèles

* et ** : flops pour le training : forward + backward, avec leurs tokenizer respectifs

** : en utilisant le maximum de tokens du contexte

GPU	RTX 4000	RTX 4060	RTX 3060
TFLOPS (FP16)	26.73	15.11	12.74

TABLE 2 – Comparaison des TFLOPS sur les GPU utilisés (en FP16 pour nos modèles)

model	GPU	#epochs	#params	dataset	train time	CE	PPL	top1	top5
1.6M	RTX 3060	14	1.6M	100	28m 21s	0.7228	2.06	0.8222	0.8663
1.6M_ext	RTX 4000	14	1.6M	100	27m 32s	0.7063	2.0265	0.8246	0.8694
1.6M_1K	RTX 4060	14	1.6M	1'000	7h 0m	0.5689	1.766	0.8587	0.8941
5.5M	RTX 4000	14	5.5M	100	29m 38s	0.6789	1.9717	0.8302	0.8739
5.5M_ext	RTX 3060	14	5.5M	100	2h 15m	0.6808	1.9755	0.8318	0.8751
5.5M_1K	RTX 4000	14	5.5M	1'000	8h 30m	0.5459	1.7260	0.8637	0.8991
5.5M_1K	RTX 4000	20	5.5M	1'000	12h 5m	0.5428	1.7208	0.8642	0.8997
11.8M_1K	RTX 4060	14	11.8M	1'000	2day 10h	0.5455	1.7256	0.8631	0.8991
20.5M	RTX 4060	14	20.5M	100	4h 14m	0.6539	1.9230	0.8391	0.8787

TABLE 3 – Comparaison des différentes valeurs des métriques sur les différents modèles.

On peut remarquer sur la Table 1 que la dimension et la contexte size du modèle influent sur le Mflops. De même cela influent aussi sur le Gflops mais significativement plus concernant la contexte size.

La Table 3 est un récapitulatif de nos entraînements avec différentes métriques sur nos différents modèles.

Nous avons cherché à entraîner différents modèles pour voir si des paramètres en entrée influait sur les différentes métriques. On peut constater avant tout que plus la taille du dataset est grande, plus l'entraînement sera long. De même, la comparaison entre le modèle 1.6M et le modèle 20.5M montre que plus un modèle a de paramètres, plus l'entraînement est long aussi. On peut s'apercevoir sur le modèle 5.5M que le GPU utilisé pour l'entraînement influe la durée aussi : la durée d'entraînement est bien plus longue sur le 5.5M_ext que 5.5M. De plus, la différence de temps d'entraînement entre le modèle 1.6M et le modèle 1.6M_ext est similaire, malgré le fait que le 1.6M soit entraîné sur un GPU moins performant que le 1.6M_ext. Cela signifie que la taille du contexte a aussi une importance sur la durée d'entraînement du modèle.

Pour l'analyse des résultats, nous cherchons à ce que le CE atteigne la plus petite valeur possible pour que le modèle prédise au mieux les tokens du SVG, que le PPL soit le plus proche de 1 pour que le modèle choisisse le bon token, et de même que le top 1 et top 5 soit au plus proche de 1 pour vérifier si le modèle prédit bien les bons tokens.

On peut s'apercevoir dans un premier temps que augmenter le contexte size du modèle augmente légèrement les performances du modèle. En revanche on voit une différence significative par rapport

au nombre de dataset. Augmenter les paramètres semblent améliorer aussi les performances mais ce très légèrement. En revanche augmenter la contexte ne semble pas trop changer les métriques. Le modèle 5.5M.1K est entraîné sur plus d'époques que les autres, les résultats semblent montrer que l'entraîner sur plus d'époques semble pas forcément pertinent : quasiment les mêmes résultats avec le même modèle sur 14 époques mais aussi avec le modèle 11.8M.1K alors que celui-ci a pris bien plus de temps à entraîner. Il y a donc un juste milieu à trouver : plus le modèle est grand plus il est probable qu'il va avoir de meilleurs résultats. En revanche le temps d'entraînement devient de plus en plus conséquent, sachant qu'à partir d'un certain modèle, ça ne semble pas pertinent d'augmenter la taille du modèle (comme entre le 5.5M et le 11.8M).

6.2 Résultats SVG

Les résultats obtenus lors de nos générations de SVG sont plus ou moins bon en fonction de la variation des différents paramètres comme la température T ou le top_k k . La température modifie la distribution de probabilité, cela veut dire que si on a $T < 1$ alors on aura une représentation assez fidèle à l'original avec un SVG assez propre. De l'autre côté si $T > 1$ alors on aura une représentation plus surprenante mais avec des risques de produire des SVG invalides. De son côté le top_k limite le nombre de tokens candidats à chaque étapes, par exemple si $k = 5$ seul les 5 tokens les plus probables sont considérés et les autres sont ignorés. De ce fait si on augmente le nombre de k alors on aura plus de diversité mais aussi plus de chance d'erreur et au contraire si on diminue le nombre de k alors on aura une génération plus prévisible. Ces deux paramètres se combinent pour définir un espace de génération.

6.2.1 Dataset de 500 œuvres

Lors de nos expériences sur un modèle de 5.5M de paramètres entraîné sur 100 images SVG et sur 5 époques, quatre configurations ont été évaluées sur chaque époques, ci-dessous sont les exemples de l'époque 5. La configuration la plus contrainte ($T = 0.5$, $k = 1$), l'image 9a produit une génération entièrement déterministe : le modèle reproduit systématiquement les structures les plus fréquentes du corpus d'entraînement sans aucune variabilité. La configuration équilibrée ($T = 0.7$, $k = 5$), l'image 9b, génère des SVG valides et structurellement cohérents tout en introduisant une légère variabilité dans les positions et dimensions des formes. La configuration intermédiaire ($T = 1.2$, $k = 20$), l'image 10a, produit des formes restent reconnaissables mais avec un peu de diversité avec certaines lignes. Enfin, la configuration la plus permissive ($T = 1.5$, $k = 50$), l'image 10b, génère des SVG avec une grande variété de formes, des lignes un peu aléatoires qui présentent une certaine incohérence. Ces résultats montrent que température et top_k doivent être ajustés conjointement : augmenter l'un sans l'autre peut mener à des comportements extrêmes (uniformité totale ou bruit pur), tandis qu'un réglage coordonné permet de naviguer entre fidélité au dataset et créativité générative. On remarque donc que notre meilleure image générée est la 9b avec $T = 0.7$ et $k = 5$.

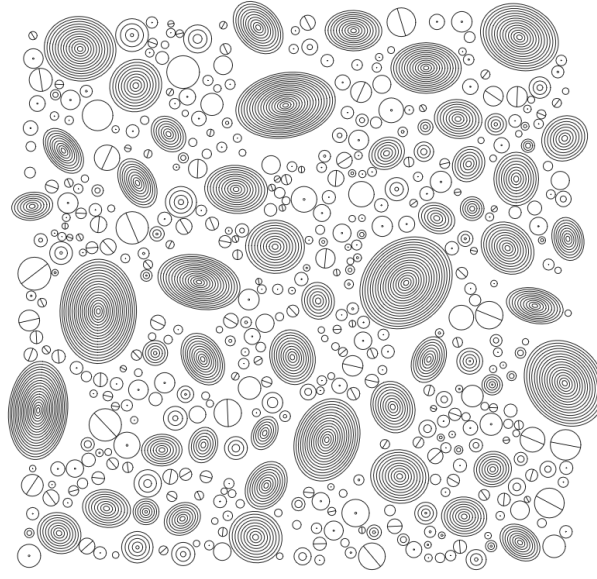
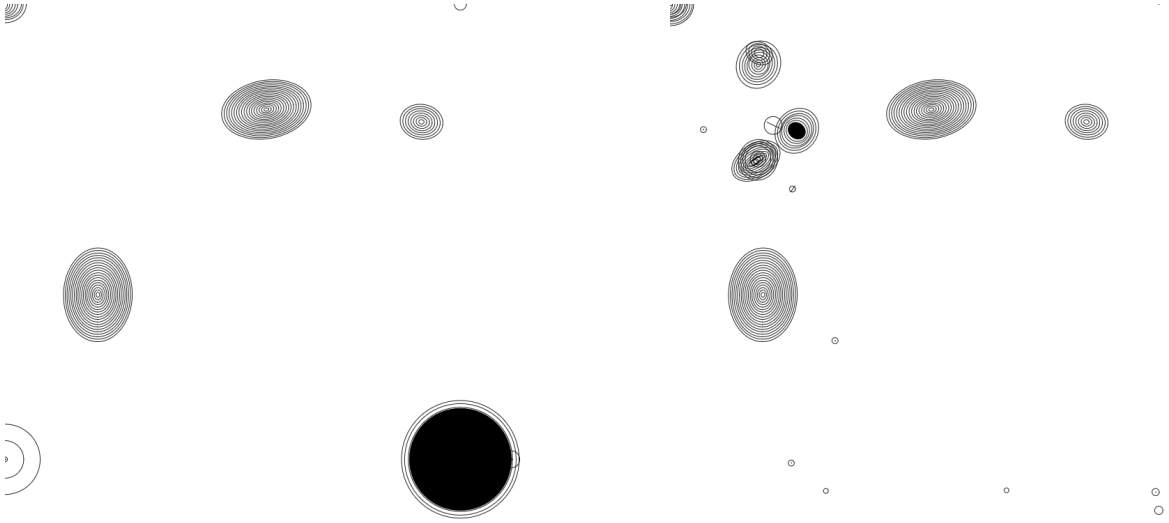


FIGURE 8 – Image de base utilisé pour la génération



(a) $T=0.5$ et $k=1$

(b) $T=0.7$ et $k=5$

FIGURE 9 – Images générés sur le modèle 5.5M avec le dataset 500 (1)

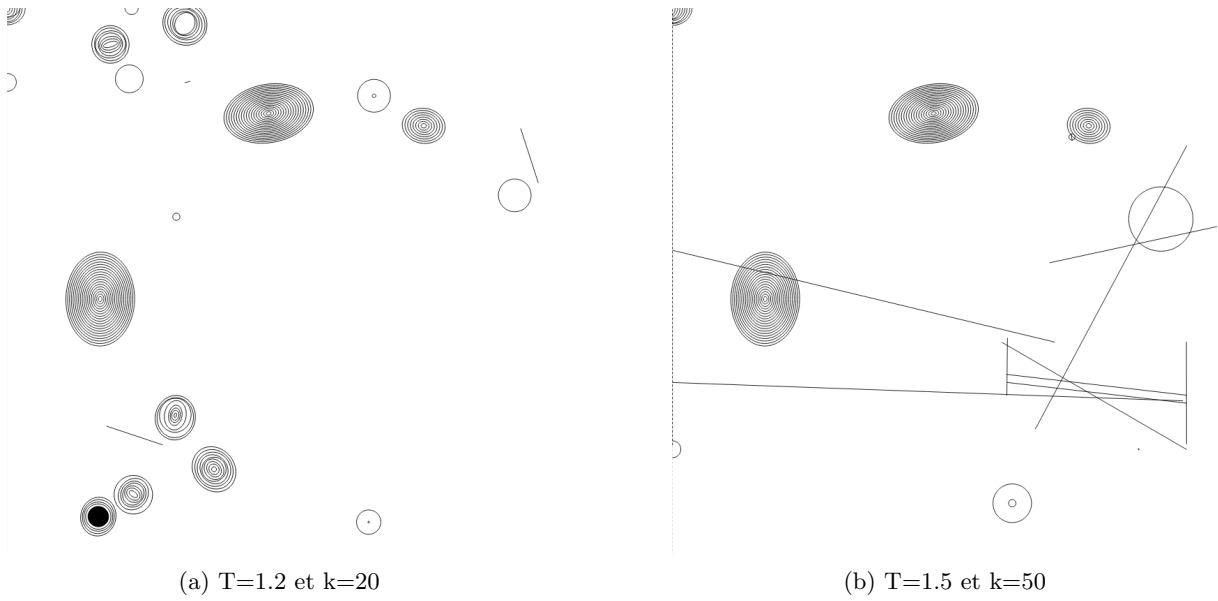


FIGURE 10 – Images générés sur le modèle 5.5M avec le dataset 500 (2)

6.2.2 Dataset de 1000 oeuvres

Nous avons gardé le modèle 5.5M de paramètres mais cette fois ci, il est entraîné sur le grand dataset qui contient 1000 images SVG. Nous avons fait des générations par rapport à l'image de base 8 mais aussi des générations en partant de rien. Nous obtenons des images qui sont assez intéressantes comme on peut le voir ci-dessous :

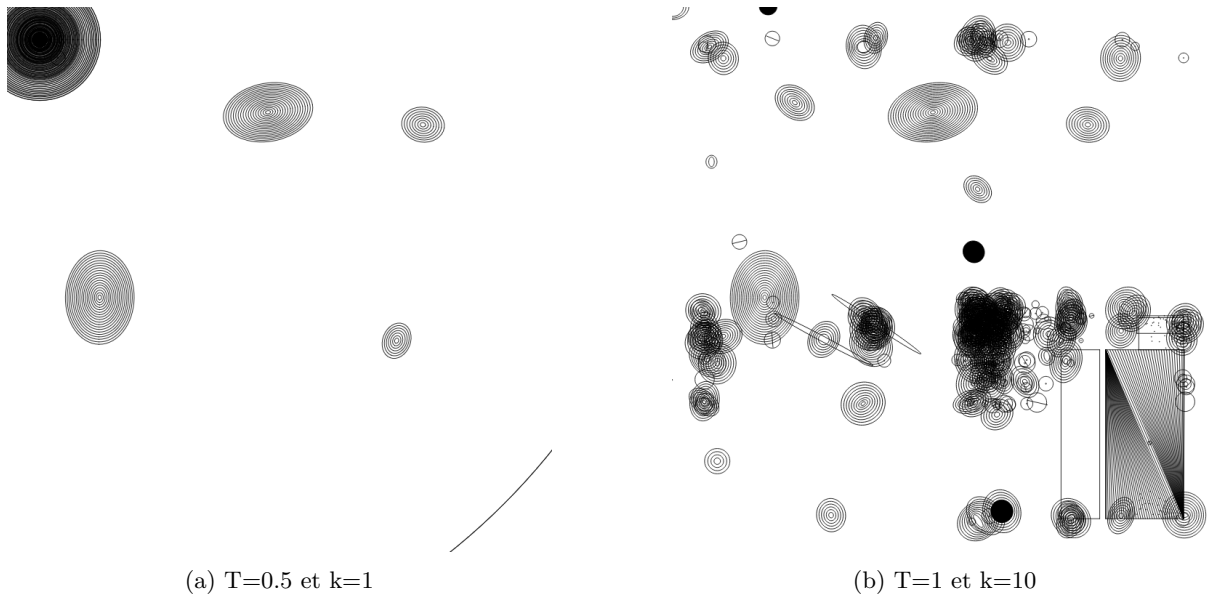
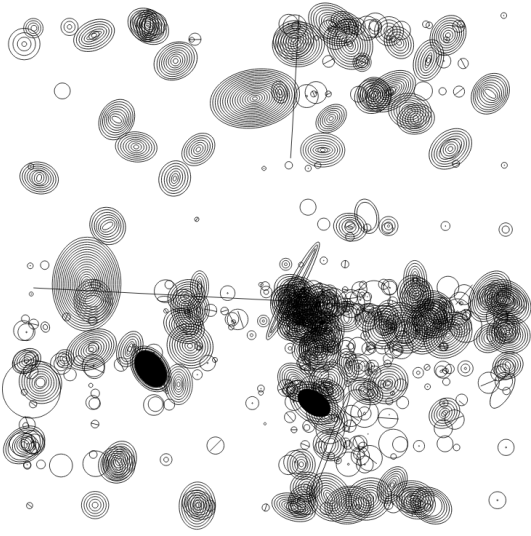
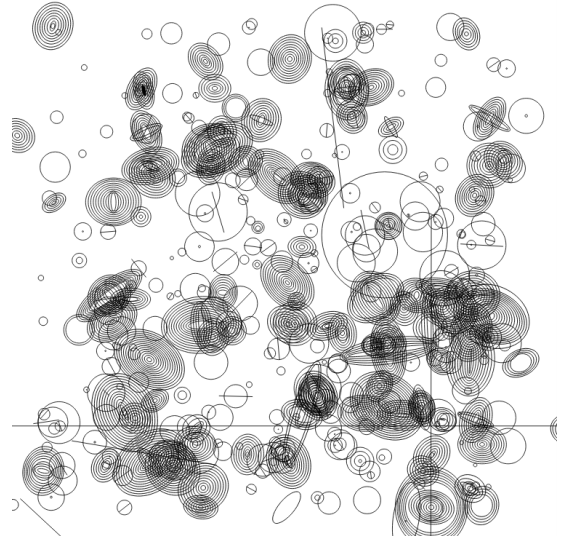


FIGURE 11 – Images générés sur le modèle 5.5M avec le dataset 1000 et l'image de base 8 (1)

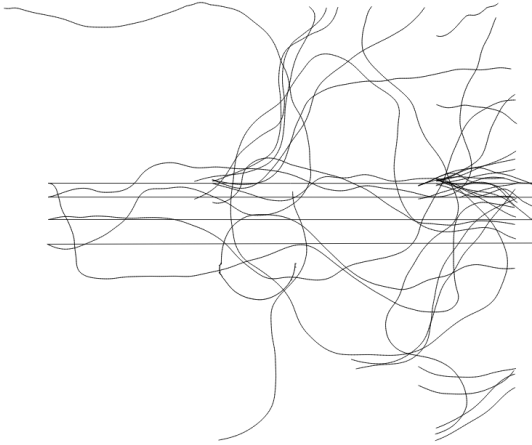


(a) $T=1.2$ et $k=20$

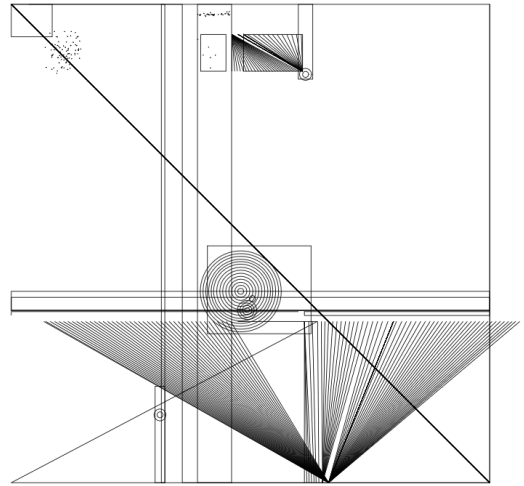


(b) $T=1.5$ et $k=50$

FIGURE 12 – Images générés sur le modèle 5.5M avec le dataset 1000 et l'image de base 8 (2)



(a) $T=0.5$ et $k=1$



(b) $T=1$ et $k=10$

FIGURE 13 – Images générés sur le modèle 5.5M avec le dataset 1000 à partir de rien (1)

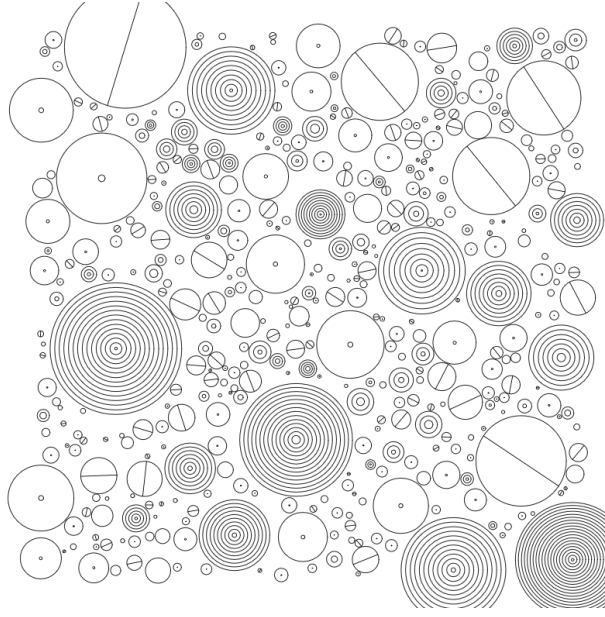
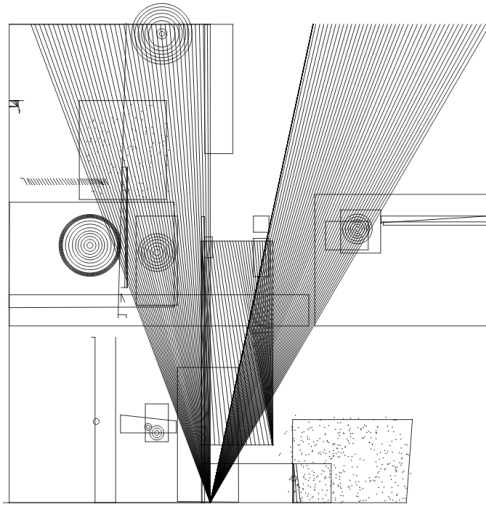
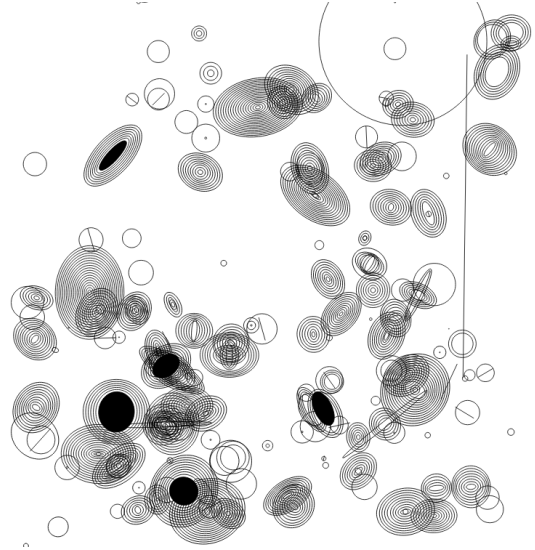


FIGURE 15 – Image de base utilisé pour la génération de G-Code



(a) $T=1.2$ et $k=20$



(b) $T=1.5$ et $k=50$

FIGURE 14 – Images générés sur le modèle 5.5M avec le dataset 1000 à partir de rien (2)

6.3 Résultats G-Code

Nous avons repris les mêmes paramètres de température et de top.k comme on a vu précédemment pour les SVG.

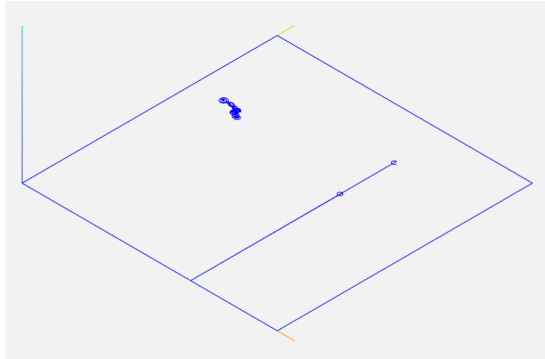
Le G-Code est un langage de commande numérique qui permet de contrôler les déplacements d'une machine. Il existe deux modes de déplacement : le mode absolu et le mode relatif. Les résultats ci-dessous ont été générés en prenant l'image de référence 15. Comme nous obtenons des générations en code G-Code, nous utilisons un visualiser en ligne qui est NCViewer [11] afin d'avoir une image qui est une représentation 3D.

On a remarqué que plus la température augmente plus la génération a de chance de s'arrêter avant la fin du temps donné et en donnant le token de fin, ce qui fait que l'on a de meilleur résultats avec des température plus basse que ce soit en G-Code absolu ou relatif.

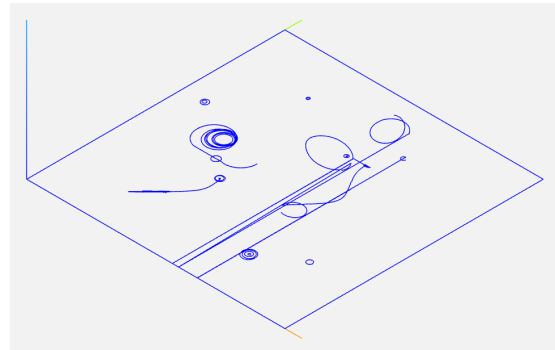
6.3.1 G-Code absolu

En mode absolu (G90), les coordonnées de chaque déplacement sont exprimées par rapport à l'origine fixe du repère machine.

Les résultats obtenus suite à l'entraînement d'un modèle de 6M de paramètres en G-Code absolu et en fonction du dataset de 500 œuvres sont les suivants :

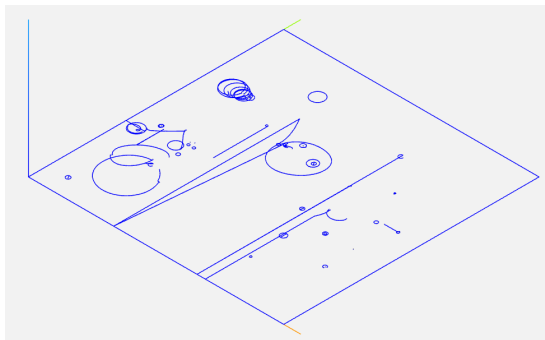


(a) $T=0.5$ et $k=1$

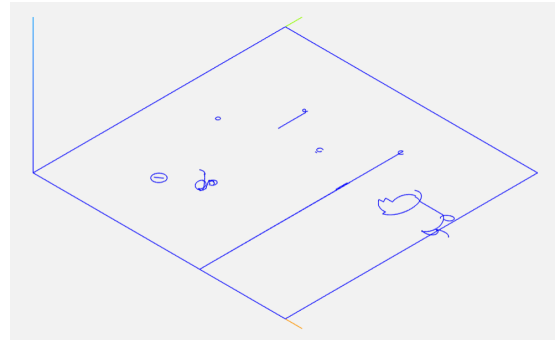


(b) $T=1$ et $k=10$

FIGURE 16 – Images générés sur le modèle 6M G-Code absolu (1)



(a) $T=1.2$ et $k=20$



(b) $T=1.5$ et $k=50$

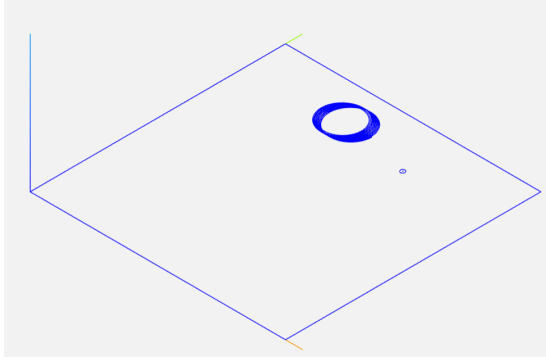
FIGURE 17 – Images générés sur le modèle 6M G-Code absolu (2)

On remarque que les générations 16b, 17a et 17b partent un peu dans tous les sens en dessinant des lignes. Alors que la figure 16a produit plus de cercles, certes assez petits, et a essayé de dessiner le diamètre du cercle comme dans l'image de base donnée 15.

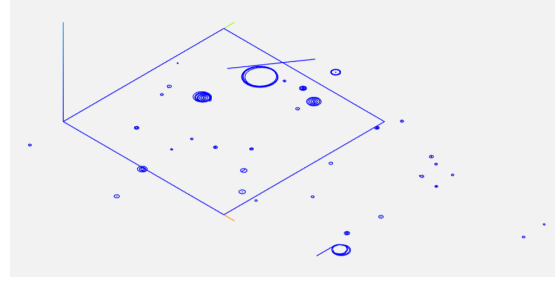
6.3.2 G-Code relatif

En mode relatif (G91), les coordonnées sont exprimées par rapport à la position actuelle de la tête.

Les résultats obtenus suite à l'entraînement d'un modèle de 6M de paramètres en G-Code relatif et en fonction du dataset de 500 œuvres sont les suivants :

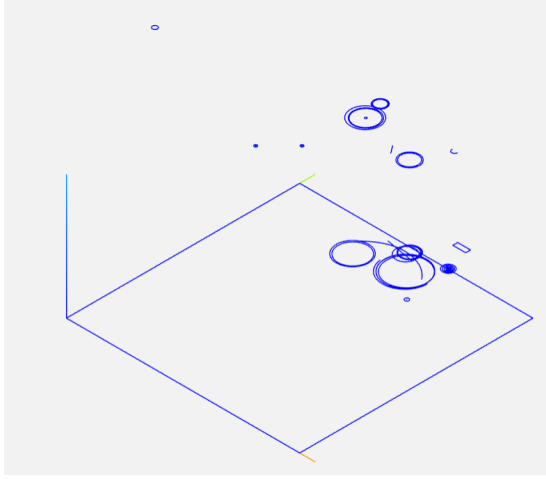


(a) $T=0.5$ et $k=1$

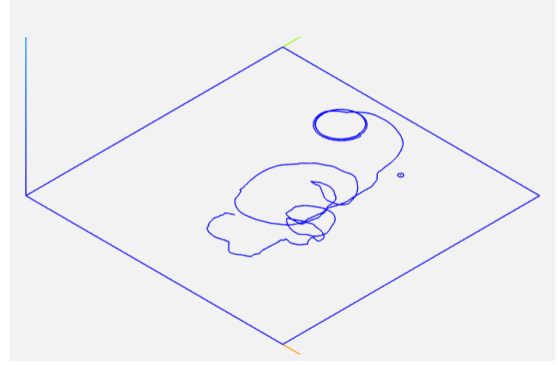


(b) $T=1$ et $k=10$

FIGURE 18 – Images générés sur le modèle 6M G-Code relatif(1)



(a) $T=1.2$ et $k=20$



(b) $T=1.5$ et $k=50$

FIGURE 19 – Images générés sur le modèle 6M G-Code relatif (2)

On remarque que les images générées en mode relatif sont meilleures et un peu plus fidèle à l'image de base 15. Les cercles ne sont pas forcément dans le carré de base mais cela est dû au mode relatif comme expliqué juste avant.

L'image 18b génère pas mal de petit cercle, mais dans l'ensemble le modèle a dû mal à dessiner les diamètres dans les cercles. On remarque même quand dans l'image 19b, le dessin n'est pas simplement des cercles et/ou des lignes mais plus un mélange des deux qui peut nous faire penser à quelqu'un qui juste tient un stylo et que la feuille bouge.

7 Conclusion

L'objectif de ce projet était d'explorer la capacité d'un modèle de langage à générer des œuvres d'art algorithmique directement au format SVG et G-code, afin de combiner le contrôle structurel avec une génération directe d'images. Cet objectif a été atteint : nous avons réussi à entraîner un modèle capable de produire des fichiers SVG syntaxiquement valides, ainsi que des séquences de G-code exploitables.

Les résultats obtenus côté SVG montrent que l'architecture NanoChat est capable d'apprendre la syntaxe et les structures récurrentes du format SVG. Les résultats en G-code sont plus difficiles à évaluer visuellement, mais indiquent quand même que le modèle arrive à générer des séquences d'instructions cohérentes. Nous avons cependant identifié plusieurs limites durant ce projet, la contrainte principale étant le temps, avec des entraînements de longue durée sur GPU qui ont limité le nombre d'expériences réalisables.

Néanmoins, ce projet montre le potentiel des modèles de langage dans des domaines non conventionnels. En effet, l'intersection entre l'intelligence artificielle et la création artistique algorithmique est peu explorée dans les cours universitaires. C'est précisément cette intersection qui a suscité notre intérêt et motivé notre choix de projet.

Références

- [1] ADOBE. *Art et algorithmes : qu'est-ce que l'art algorithmique ?* URL : <https://www.adobe.com/fr/creativecloud/illustration/discover/digital-art/algorithmic-art.html>.
- [2] URL : <https://apxml.com/tools/vram-calculator>.
- [3] Weights BIASES. URL : <https://wandb.ai/>.
- [4] DEEPSVG. *DeepSVG : A Hierarchical Generative Network for Vector Graphics Animation*. 2020. URL : <https://arxiv.org/pdf/2007.11301>.
- [5] Hugging FACE. URL : <https://huggingface.co/OmniSVG/OmniSVG>.
- [6] FREEPIK. URL : <https://fr.freepik.com/>.
- [7] FREEPIK. *How to generate images with AI*. URL : <https://www.youtube.com/watch?v=voEEBB4qSxI>.
- [8] Andrej KARPATY. *NanoChat*. 2025. URL : <https://github.com/karpathy/nanochat>.
- [9] Fahd MIRZA. *Revolutionizing Graphic Design : Introducing OmniSVG*. URL : <https://www.youtube.com/watch?v=lKehrYAhTM8>.
- [10] MUTEK. *L'art génératif : nouveau vecteur de création*. 2024. URL : <https://www.adobe.com/fr/creativecloud/illustration/discover/digital-art/algorithmic-art.html>.
- [11] NCVIEWER. URL : <https://ncviewer.com/>.
- [12] PROCESSING. URL : <https://processing.org/>.
- [13] *Punkerico*. URL : <https://icon.punkerduck.com/>.
- [14] RECRAFT.AI. URL : <https://www.recraft.ai/>.
- [15] ROBOSVG. *ROBOSVG : A UNIFIED FRAMEWORK FOR INTERACTIVE SVG GENERATION WITH MULTI-MODAL GUIDANCE*. 2025. URL : <https://arxiv.org/pdf/2510.22684>.
- [16] SVGVIEWER. URL : <https://www.svgviewer.dev/>.
- [17] Roman VEROSTKO. *ALGORITHMIC ART*. 1994. URL : <https://www.verostko.com/algorithm.html>.
- [18] URL : <https://github.com/plottertools/vpype-gcode>.
- [19] YESCHAT.AI. URL : <https://www.yeschat.ai/gpts-9t557DQlkVm-G-Code-Creator>.